

SwarmState Technical Specification v2.0

grave0x

2026-09-01

Contents

1	Introduction	3
1.1	Core Principles	3
2	Component Status	3
3	System Architecture	3
3.1	Kernel	4
3.2	Kernel Operations	5
4	Registry and Learning	5
4.1	Key Structure	5
4.2	Learning Loop	5
4.3	Cross-Node Propagation and Integrity	5
5	Orchestrator and Plan Generation	6
5.1	Flow	6
5.2	Retry and Rescue	6
5.3	The Central Loop	7
6	Mesh and Federation	7
6.1	Transports	7
6.2	Mesh Message Types	7
6.3	Pools and Bridges	7
6.4	Inference Broker	8
7	Governance and Safety	8
7.1	Principles	8
7.2	Authorization Flow	9
7.3	Multi-Party Approval and Audit	9
8	Support and Operations	9
9	User Interface	10
9.1	Widget Catalogue	10
9.2	Role Views	10
10	Security Posture	10
11	Acceptance Criteria	11

12 Roadmap and Milestones	11
13 Conclusion	11

1 Introduction

SwarmState is a local-first, self-improving edge intelligence platform. It couples a deterministic C kernel with a registry-driven orchestrator, a peer-to-peer mesh, and a modular UI kit to provide constrained, private, and low-cost AI-assisted operations. The system is designed for offline and remote environments, with optional cloud escalation only when explicitly enabled.

This document is the executive and reference overview. The full consolidated design is `spec.md` (Master Technical Specification v2.0); detailed component specs live in `docs/`. Statuses below are as built on 2026-09-01.

1.1 Core Principles

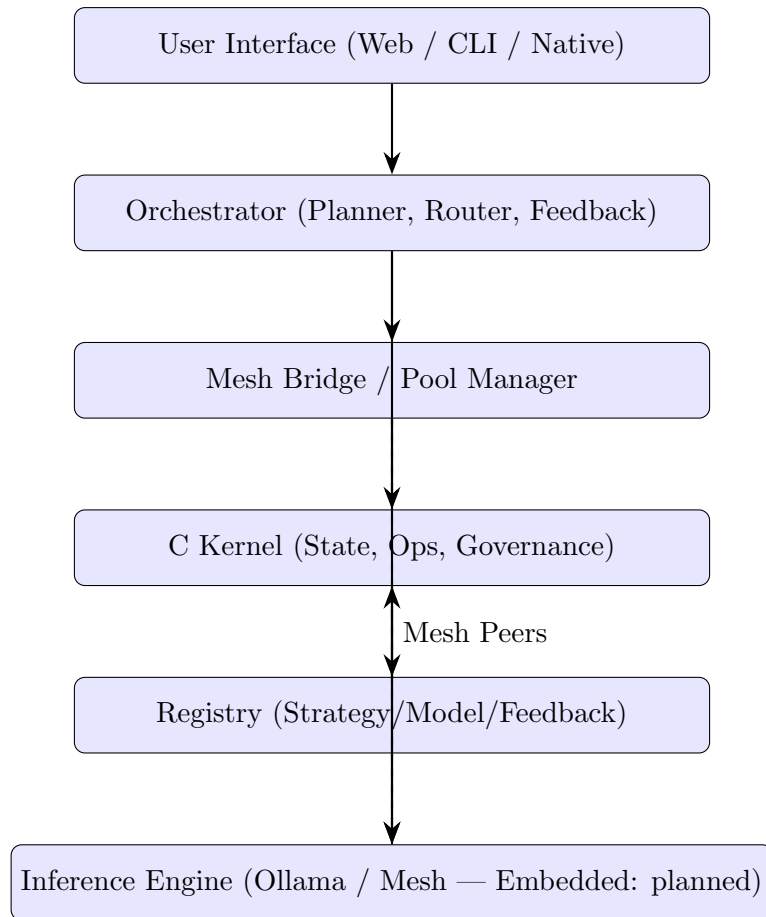
- **Mechanical containment:** The kernel enforces safety rules in C, not through prompts.
- **Context is contained:** The LLM sees only summaries, diffs, and plans—never the full state (62-byte prompts on routine tasks).
- **Semantic feedback:** Oracles judge task success, and the registry learns from those outcomes.
- **Default-deny mesh:** Nodes and pools must be explicitly trusted.
- **Human governance:** High-risk actions require authorization and recorded reasons.

2 Component Status

Component	Status	Notes
C Kernel	Shipped	Ops, governance gate, path safety; 175 C checks
Registry	Shipped	Semantic, temporal, per-pool partitions
Orchestrator	Shipped	Route→see→diagnose→rescue→learn loop
Mesh	Shipped	TCP + Tailcat + LoRa (sim); pools, bridges, multi-homing
Governance	Shipped	Auth tokens, recorded reasons, multi-party, audit ledger
Windows Node	Shipped	Self-contained zip (node exe + kernel DLL + web UI)
UI Kit	Partial	v1 shell + 6 core widgets; full catalogue planned
Identity / Roles	Partial	Signed identity store (DORMANT); role gating on roadmap
Embedded Inference	Planned	llama.cpp integration (spec v1)
Support / Ops Layer	Planned	MSP dashboard, Watch mode
Native App	Planned	Standalone-first app
Radio Backbone	Planned	Licensed point-to-point / LMR

3 System Architecture

The system is composed of six logical layers:



3.1 Kernel

The kernel is a C library and daemon that owns the local state and executes plans. It exposes a small, typed set of operations and enforces:

- Path containment via `realpath` and `O_NOFOLLOW`
- Binary allowlists and argument validation for `EXECUTE`
- Schema validation for all incoming plans
- Governance checks for high-risk ops (token + reason)

3.2 Kernel Operations

Op	Risk	Purpose
SS_OP_READ	Low	Read file contents (optional line range)
SS_OP_WRITE	Medium	Write file contents (blind-write guard applies)
SS_OP_GREP	Low	POSIX ERE search over a path
SS_OP_DIFF	Low	Git diff (repo or one path)
SS_OP_STATUS	Low	Git status –porcelain
SS_OP_EXECUTE	High	Whitelisted binary, filtered args (allowlist)
SS_OP_AST_PARSE / SS_OP_AST_QUERY	Low	Tree-sitter parse / query
SS_OP_SYMBOL_SUMMARY	Low	Doc-layer symbol summaries (SYMBOLIC strategy)
FILE_REQUEST / MESH_SEND	Medium	Mesh/bridge layer: serve file, relay op (bridge policy)

4 Registry and Learning

4.1 Key Structure

A persistent key-value store that tracks success rates per task signature, strategy, model, and resource context. Keys follow a deterministic pattern:

Key	Meaning
STRAT:<plan-sig>:<strategy>	Strategy success rate
STRAT:<plan-sig>:<strategy>:<model>	Per-model strategy success
MODEL:q:<query-sig>:<model>	Query-level model routing
POOL:<pool-id>:STRAT:<sig>:<strategy>	Cross-pool learning partition (source-tagged)
<key> value	(fb_ok, fb_count) plus timing stats
temporal tags	min / max / last_ts freshness window

4.2 Learning Loop

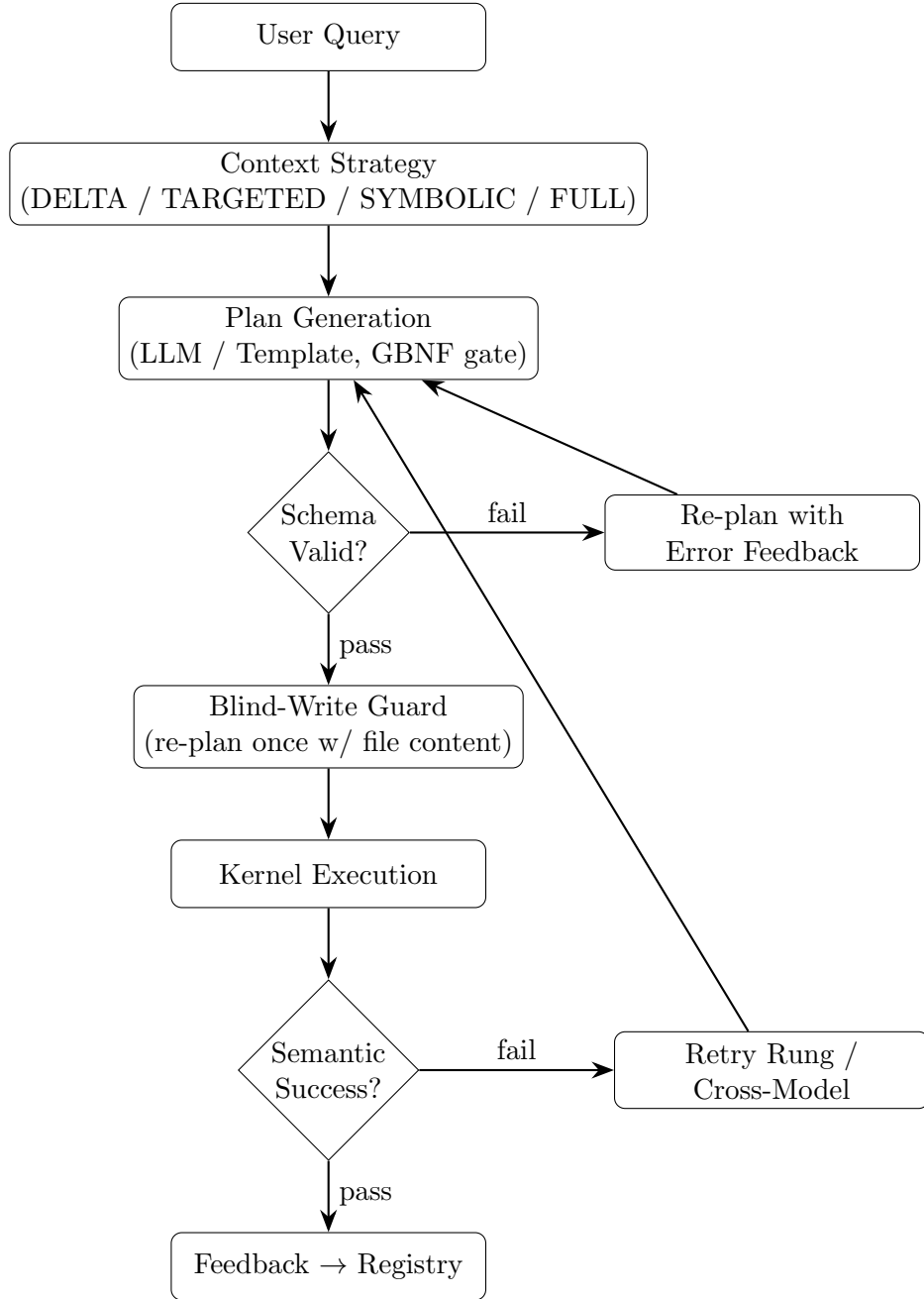
1. User query → task signature computed.
2. Router checks the registry for a proven strategy/model.
3. Plan generated and executed.
4. Oracle scores the semantic outcome.
5. Feedback written to the registry (local and bridged).

4.3 Cross-Node Propagation and Integrity

Mesh ops carry registry updates. Bridged learning is source-tagged (POOL:<src>:...) and never pollutes the local partition. **Planned:** a hash-chain journal with node-signed entries makes poisoning detectable (see §10).

5 Orchestrator and Plan Generation

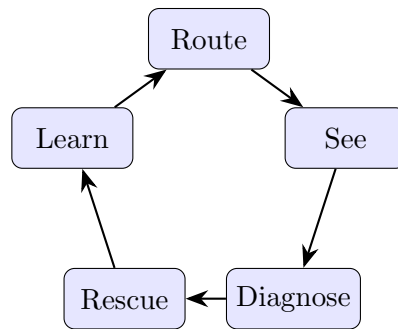
5.1 Flow



5.2 Retry and Rescue

- Schema failure → re-plan with validation errors fed back.
- Execution failure → re-plan with the kernel error.
- Both fail → cross-model retry (a different brain).
- Still failing → escalate to cloud or a human.

5.3 The Central Loop



6 Mesh and Federation

6.1 Transports

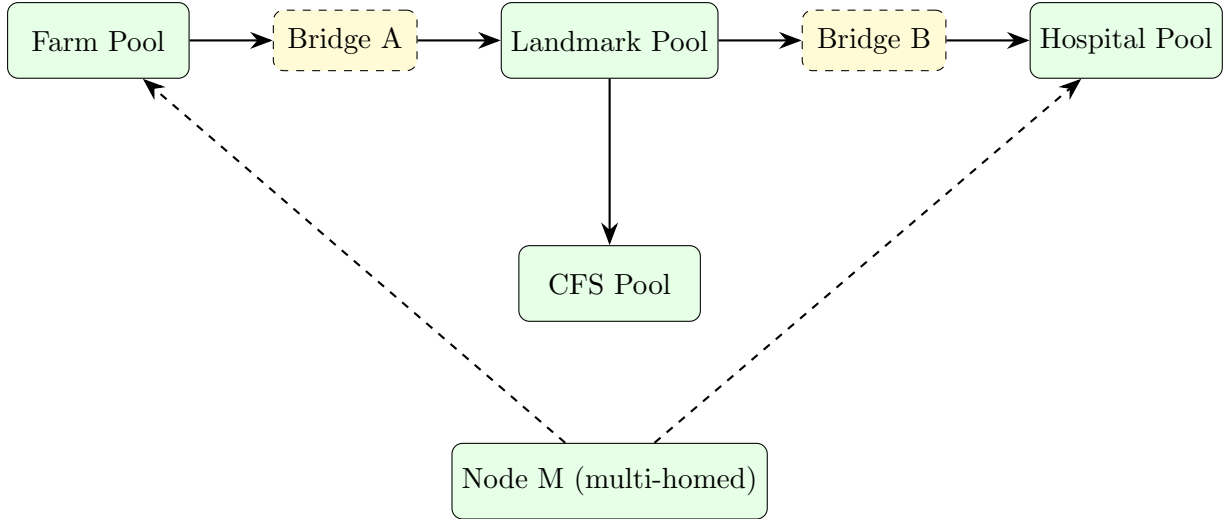
Transport	Status	Use Case
TCP	Shipped	LAN
Tailcat	Shipped	Encrypted WAN without control plane (WireGuard + DERP)
LoRa	Partial (sim)	Long-range, low-bandwidth; real hardware on roadmap
Point-to-point radio	Planned	Licensed backbone / LMR

6.2 Mesh Message Types

Implemented on the node daemon (`meshd.py`): `hello`, `join`, `join.ok`, `join.denied`, `member.added`, `capability`, `req`, `resp`, `file.request`, `file.response`, `lora.share`, `op`. Spec'd for the full vision: `heartbeat`, `capability.announce`, `inference.request`, `inference.response`, `state.diff`.

6.3 Pools and Bridges

- **Pool**: a set of nodes sharing a trust boundary.
- **Bridge**: a scoped connection between two pools. Policy scopes allowed message types (default: registry learning only), task-signature globs, TTL, and data minimization (MODEL routing keys stay local). Both sides must accept.
- **Multi-homing**: a node may belong to several pools, each with its own registry partition and identity.



6.4 Inference Broker

Provider nodes broadcast capability announcements (model, load, freshness). Consumers send inference requests; the broker scores candidates and routes to the best provider, with fallback to local inference when no provider is available.

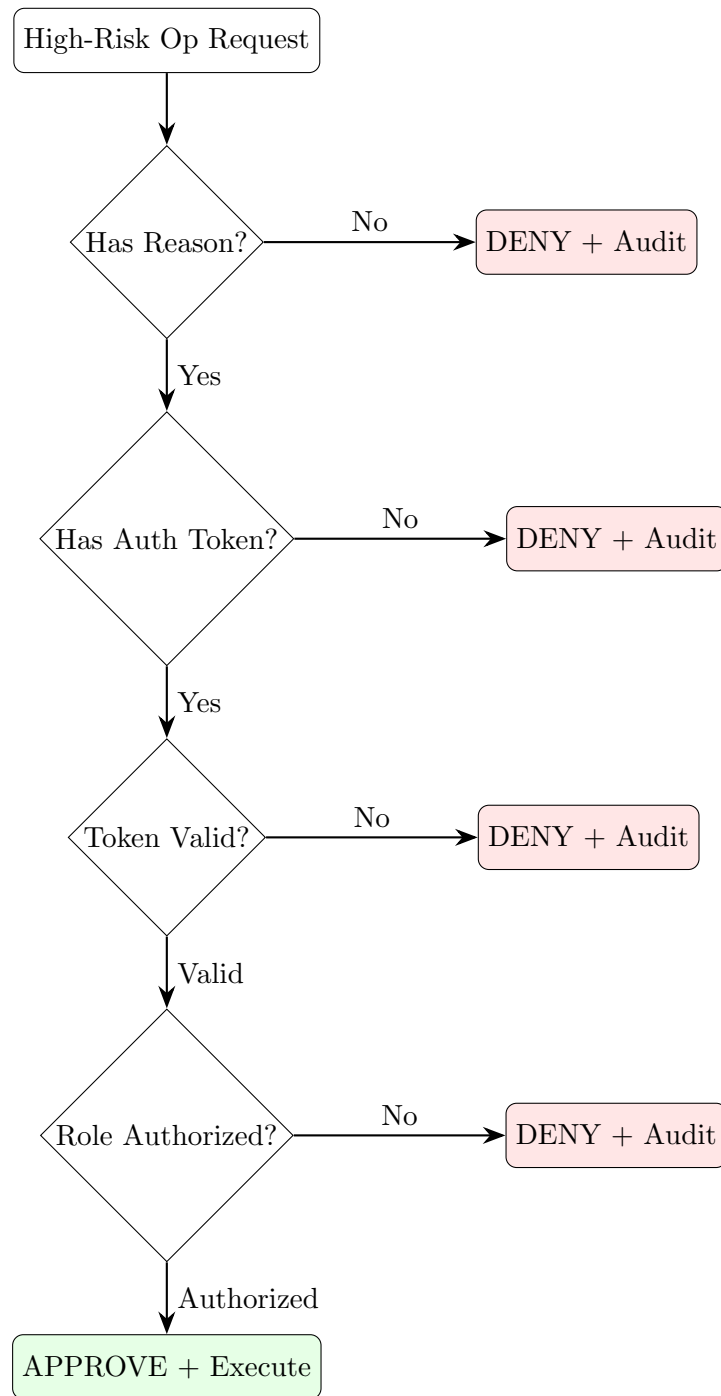
7 Governance and Safety

7.1 Principles

- The model never acts directly.
- High-risk ops require authorization.
- All decisions are audited.

Risk tiers: SAFE (read-only), WRITE, EXEC, ACTUATE (reserved for future **CONTROL_*** ops). Governed ops need a valid HMAC-SHA256 token (**ss:<role>:<expiry>**, short TTL, constant-time verify) *and* a recorded reason. The reason is checked first, so the denial tells the user what to fix.

7.2 Authorization Flow



7.3 Multi-Party Approval and Audit

For cross-pool actions, every required authority must present a valid token; the gate checks all roles and always audits. Every approval/denial is appended to the ledger with timestamp, user, role, op, and reason. The ledger is append-only; hash-chain tamper evidence is planned (see §10).

8 Support and Operations

Role-based access control enables MSP-style management of remote nodes:

Role	Scope	Typical Actions
Field	Single device	Use assistant, view tasks, request help
Support (MSP)	Managed client pool	Diagnostics, rejoin, invite, escalate
Architect	Full system	Updates, rollback, audit, cross-pool
Watch	Automatic	Health checks, safe self-remediation

Watch mode: each node monitors its own health (disk, memory, CPU, registry, mesh). Deviations trigger alerts; a finite set of safe auto-fixes may run (restart link, rejoin pool, resend ops)—all logged and reversible. **Prompt-based administration** translates natural-language technician commands into structured, kernel-validated ops. Status: roles and flows are specified; enforcement (role gating, Watch) is on the roadmap.

9 User Interface

The UI is a web app served from the local node: a widget shell plus role-based widgets.

9.1 Widget Catalogue

Map, Task List, Alert Banner, Team Roster, Vitals/Sensor Panel, Governance Console, Voice Input, Knowledge Search, QR/Invite, Status Dashboard.

9.2 Role Views

Role	Typical Widgets
Field Worker	Map, Task List, Alert Banner, Voice Input
Supervisor	Roster, Task List, Governance Console, Alerts
Admin	QR/Invite, Status Dashboard, Knowledge Search
Observer	Map, Alerts, Knowledge Search (read-only)

Layout is config-driven (`config/widgets.json`): a non-technical user sees a role-appropriate dashboard, and the layout changes by editing config, not code. Governance is enforced at the node regardless of UI role.

10 Security Posture

A 67-case attack battery (kernel exec/write containment, governance, multi-party, pool/join, mesh) finished 61 PASS / 5 WEAK / 0 FAIL, with single-use invites (replay protection), reason-before-auth error paths, backslash path handling, and a registry freshness window applied. Findings: `docs/redteam-findings.md`.

Remaining Gap	Mitigation Plan
Registry tampering	Hash-chain journal with node signatures
LoRa encryption	AES-256-GCM with pre-shared or exchanged keys
Pool secret storage	OS keychain / DPAPI
Windows EXECUTE	PowerShell allowlist or service manager
External pen test	Schedule before production pilot

11 Acceptance Criteria

Capability	Result	Notes
C test suite	175 checks	<code>make test</code> ; full suite adds harness/py/rg
Red-team battery	67 cases	61 PASS / 5 WEAK / 0 FAIL, PITCH_READY
3-node live acceptance	PASS	Real Ollama + strict grammar gate
Contained prompts	~62 B	vs. naive 100+ KB prompts
Grep 10k files	< 15 ms	auto 0.81 ms, 18x headroom
Batch latency	133 μ s / 5 ops	26.7 μ s per op; MVP bar < 1 ms
Mesh convergence	2 nodes $\rightarrow$ 1 hash	2 STRAT each way, tail 4, ~1.2 MB RSS

12 Roadmap and Milestones

- **Short term (2–4 weeks):** embedded inference engine (CPU-only first); pitch demo on a clean machine; Windows EXECUTE implementation.
- **Medium term (1–3 months):** first pilot (CFS or MSP); external security review; native app alpha; registry hash-chain integrity.
- **Long term (3–12 months):** radio backbone trial; multi-party governance in C; full support/ops layer; community contributions.

13 Conclusion

SwarmState is a complete, local-first edge intelligence platform that combines mechanical safety, semantic feedback, and peer-to-peer mesh learning. It is ready for real-world pilots in agriculture, emergency services, healthcare, and remote operations. The full consolidated reference is `spec.md` (v2.0); this document tracks the same architecture and statuses.